

```
%pip install transformers torch tf-keras
```

```
Requirement already satisfied: transformers in d:\college\year6s1\nlp-dl\.venv\lib\site-pac
Requirement already satisfied: torch in d:\college\year6s1\nlp-dl\.venv\lib\site-packages (
Requirement already satisfied: tf-keras in d:\college\year6s1\nlp-dl\.venv\lib\site-package
Requirement already satisfied: filelock in d:\college\year6s1\nlp-dl\.venv\lib\site-package
Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in d:\college\year6s1\nlp-dl\.v
Requirement already satisfied: numpy>=1.17 in d:\college\year6s1\nlp-dl\.venv\lib\site-pack
Requirement already satisfied: packaging>=20.0 in d:\college\year6s1\nlp-dl\.venv\lib\site-
Requirement already satisfied: pyyaml>=5.1 in d:\college\year6s1\nlp-dl\.venv\lib\site-pack
Requirement already satisfied: regex!=2019.12.17 in d:\college\year6s1\nlp-dl\.venv\lib\sit
Requirement already satisfied: requests in d:\college\year6s1\nlp-dl\.venv\lib\site-package
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in d:\college\year6s1\nlp-dl\.ve
Requirement already satisfied: safetensors>=0.4.3 in d:\college\year6s1\nlp-dl\.venv\lib\si
Requirement already satisfied: tqdm>=4.27 in d:\college\year6s1\nlp-dl\.venv\lib\site-packa
Requirement already satisfied: fsspec>=2023.5.0 in d:\college\year6s1\nlp-dl\.venv\lib\site
Requirement already satisfied: typing-extensions>=3.7.4.3 in d:\college\year6s1\nlp-dl\.ven
Requirement already satisfied: sympy>=1.13.3 in d:\college\year6s1\nlp-dl\.venv\lib\site-pa
Requirement already satisfied: networkx>=2.5.1 in d:\college\year6s1\nlp-dl\.venv\lib\site-
Requirement already satisfied: Jinja2 in d:\college\year6s1\nlp-dl\.venv\lib\site-packages
Requirement already satisfied: setuptools in d:\college\year6s1\nlp-dl\.venv\lib\site-packa
Requirement already satisfied: tensorflow<2.21,>=2.20 in d:\college\year6s1\nlp-dl\.venv\li
Requirement already satisfied: absl-py>=1.0.0 in d:\college\year6s1\nlp-dl\.venv\lib\site-p
Requirement already satisfied: astunparse>=1.6.0 in d:\college\year6s1\nlp-dl\.venv\lib\sit
Requirement already satisfied: flatbuffers>=24.3.25 in d:\college\year6s1\nlp-dl\.venv\lib\
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in d:\college\year6s1\nl
Requirement already satisfied: google_pasta>=0.1.1 in d:\college\year6s1\nlp-dl\.venv\lib\s
Requirement already satisfied: libclang>=13.0.0 in d:\college\year6s1\nlp-dl\.venv\lib\site
Requirement already satisfied: opt_einsum>=2.3.2 in d:\college\year6s1\nlp-dl\.venv\lib\sit
Requirement already satisfied: protobuf>=5.28.0 in d:\college\year6s1\nlp-dl\.venv\lib\site
Requirement already satisfied: six>=1.12.0 in d:\college\year6s1\nlp-dl\.venv\lib\site-pack
Requirement already satisfied: termcolor>=1.1.0 in d:\college\year6s1\nlp-dl\.venv\lib\site
Requirement already satisfied: wrapt>=1.11.0 in d:\college\year6s1\nlp-dl\.venv\lib\site-pa
Requirement already satisfied: grpcio<2.0,>=1.24.3 in d:\college\year6s1\nlp-dl\.venv\lib\s
Requirement already satisfied: tensorboard~=2.20.0 in d:\college\year6s1\nlp-dl\.venv\lib\s
Requirement already satisfied: keras>=3.10.0 in d:\college\year6s1\nlp-dl\.venv\lib\site-pa
Requirement already satisfied: h5py>=3.11.0 in d:\college\year6s1\nlp-dl\.venv\lib\site-pac
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in d:\college\year6s1\nlp-dl\.venv\l
Requirement already satisfied: charset_normalizer<4,>=2 in d:\college\year6s1\nlp-dl\.venv\
Requirement already satisfied: idna<4,>=2.5 in d:\college\year6s1\nlp-dl\.venv\lib\site-pac
Requirement already satisfied: urllib3<3,>=1.21.1 in d:\college\year6s1\nlp-dl\.venv\lib\si
Requirement already satisfied: certifi>=2017.4.17 in d:\college\year6s1\nlp-dl\.venv\lib\si
Requirement already satisfied: markdown>=2.6.8 in d:\college\year6s1\nlp-dl\.venv\lib\site-
Requirement already satisfied: pillow in d:\college\year6s1\nlp-dl\.venv\lib\site-packages
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in d:\college\year6s1\
Requirement already satisfied: werkzeug>=1.0.1 in d:\college\year6s1\nlp-dl\.venv\lib\site-
Requirement already satisfied: wheel<1.0,>=0.23.0 in d:\college\year6s1\nlp-dl\.venv\lib\si
Requirement already satisfied: rich in d:\college\year6s1\nlp-dl\.venv\lib\site-packages (f
Requirement already satisfied: namex in d:\college\year6s1\nlp-dl\.venv\lib\site-packages (
Requirement already satisfied: optree in d:\college\year6s1\nlp-dl\.venv\lib\site-packages
Requirement already satisfied: mpmath<1.4,>=1.1.0 in d:\college\year6s1\nlp-dl\.venv\lib\si
Requirement already satisfied: colorama in d:\college\year6s1\nlp-dl\.venv\lib\site-package
Requirement already satisfied: markupsafe>=2.1.1 in d:\college\year6s1\nlp-dl\.venv\lib\sit
Requirement already satisfied: markdown-it-py>=2.2.0 in d:\college\year6s1\nlp-dl\.venv\lib
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in d:\college\year6s1\nlp-dl\.venv\l
Requirement already satisfied: mdurl~=0.1 in d:\college\year6s1\nlp-dl\.venv\lib\site-packa
Note: you may need to restart the kernel to use updated packages.
```

Bài 1: Khôi phục Masked Token (Masked Language Modeling) Trong tác vụ này, chúng ta sẽ che một vài từ trong câu bằng một token đặc biệt (thường là [MASK]) và yêu cầu mô hình dự đoán xem từ gốc là gì. Chúng ta sẽ sử dụng một mô hình thuộc họ BERT.

Yêu cầu: Sử dụng pipeline fill-mask để dự đoán từ bị thiếu trong câu sau: Hanoi is the [MASK] of Vietnam.

```

from transformers import pipeline
# 1. Tải pipeline "fill-mask"
# Pipeline này sẽ tự động tải một mô hình mặc định phù hợp (thường là một biến thể của BERT)
mask_filler = pipeline("fill-mask")
# 2. Câu đầu vào với token [MASK]
input_sentence = "Hanoi is the <mask> of Vietnam."
# 3. Thực hiện dự đoán
# top_k=5 yêu cầu mô hình trả về 5 dự đoán hàng đầu
predictions = mask_filler(input_sentence, top_k=5)
# 4. In kết quả
print(f"Câu gốc: {input_sentence}")
for pred in predictions:
    print(f"Dự đoán: '{pred['token_str']}' với độ tin cậy: {pred['score']:.4f}")
    print(f"-> Câu hoàn chỉnh: {pred['sequence']}")

```

No model was supplied, defaulted to distilbert/distilroberta-base and revision fb53ab8 ([http://huggingface.co/distilbert/distilroberta-base/resolve/main/pytorch_model.bin](#))
 Using a pipeline without specifying a model name and revision in production is not recommended
 Some weights of the model checkpoint at distilbert/distilroberta-base were not used when in
 - This IS expected if you are initializing RobertaForMaskedLM from the checkpoint of a model
 - This IS NOT expected if you are initializing RobertaForMaskedLM from the checkpoint of a
 Device set to use cpu

Câu gốc: Hanoi is the <mask> of Vietnam.
 Dự đoán: ' capital' với độ tin cậy: 0.9341
 -> Câu hoàn chỉnh: Hanoi is the capital of Vietnam.
 Dự đoán: ' Republic' với độ tin cậy: 0.0300
 -> Câu hoàn chỉnh: Hanoi is the Republic of Vietnam.
 Dự đoán: ' Capital' với độ tin cậy: 0.0105
 -> Câu hoàn chỉnh: Hanoi is the Capital of Vietnam.
 Dự đoán: ' birthplace' với độ tin cậy: 0.0054
 -> Câu hoàn chỉnh: Hanoi is the birthplace of Vietnam.
 Dự đoán: ' heart' với độ tin cậy: 0.0014
 -> Câu hoàn chỉnh: Hanoi is the heart of Vietnam.

Khó khăn:

- Mô hình được pipeline của transformer lựa chọn không sử dụng [MASK] mà sử dụng

Câu hỏi:

1. Mô hình đã dự đoán đúng từ capital không?
 - Mô hình đã dự đoán đúng từ "capital" với độ tin cậy lên đến 0.9341
2. Tại sao các mô hình Encoder-only như BERT lại phù hợp cho tác vụ này?
 - Mô hình BERT phù hợp cho tác vụ đoán từ bị mask vì BERT sử dụng mô hình MLM = Masked Language Modelling, tức là tác vụ đoán từ bị mask chính là cách BERT được train.
 - Cụ thể, các Encoder có cơ chế self-attention 2 chiều để tập trung cả hai hướng trước sau, không bị ảnh hưởng bởi thứ tự sinh token.

Bài 2: Dự đoán từ tiếp theo (Next Token Prediction) Tác vụ này yêu cầu mô hình sinh ra phần tiếp theo của một đoạn văn bản cho trước. Chúng ta sẽ sử dụng một mô hình thuộc họ GPT.

Yêu cầu: Sử dụng pipeline text-generation để sinh ra phần tiếp theo cho câu: The best thing about learning NLP is

```

from transformers import pipeline
# 1. Tải pipeline "text-generation"
# Pipeline này sẽ tự động tải một mô hình phù hợp (thường là GPT-2)

```

```

generator = pipeline("text-generation")
# 2. Đoạn văn bản mẫu
prompt = "The best thing about learning NLP is"
# 3. Sinh văn bản
# max_length: tổng độ dài của câu mẫu và phần được sinh ra
# num_return_sequences: số lượng chuỗi kết quả muốn nhận
generated_texts = generator(prompt, max_length=50, num_return_sequences=1)
# 4. In kết quả
print(f"Câu mẫu: '{prompt}'")
for text in generated_texts:
    print("Văn bản được sinh ra:")
    print(text['generated_text'])

```

No model was supplied, defaulted to openai-community/gpt2 and revision 607a30d (<https://huggingface.co/openai-community/gpt2>)
Using a pipeline without specifying a model name and revision in production is not recommended.
d:\College\year6s1\nlp-dl\.venv\Lib\site-packages\huggingface_hub\file_download.py:143: Use To support symlinks on Windows, you either need to activate Developer Mode or to run Python warnings.warn(message)

Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to Device set to use cpu

Truncation was not explicitly activated but `max\_length` is provided a specific value, please consider setting `padding` to `eos\_token\_id`:50256 for open-end generation.

Both `max\_new\_tokens` (=256) and `max\_length` (=50) seem to have been set. `max\_new\_tokens`

Câu mẫu: 'The best thing about learning NLP is'

Văn bản được sinh ra:

The best thing about learning NLP is that it's a matter of practice. In theory, it could be

Learn to Recognize Empathy

In a nutshell, you want to learn to recognize empathy or empathy for others.

In a way, empathy is just one way to see the world. It's all about feeling good about yourself

When you're feeling good about yourself, you can always feel bad about yourself. This is why

Now, you can also see how you can see both the world and yourself.

I think that we all need to learn to see each other as individuals.

I think that people need to be able to see the world as individuals, and they need to feel

I think that people need to be able to see the world as individuals, and they need to feel

To see the world as individuals is to experience a state of being in a state of being in a

To see the world as individuals is to experience a

Câu hỏi:

1. Kết quả sinh ra có hợp lý không?

- Kết quả đoạn đầu tiên hợp lý nhưng sau đó nói về Empathy có vẻ không còn hợp lý.

2. Tại sao các mô hình Decoder-only như GPT lại phù hợp cho tác vụ này?

- Decoder-only được huấn luyện theo cách dự đoán từ tiếp theo bằng các từ trước đó
- Decoder sinh tuần tự
- Decoder chỉ nhìn bên trái không phải hai chiều

Bài 3: Tính toán Vector biểu diễn của câu (Sentence Representation) Một trong những ứng dụng mạnh mẽ nhất của BERT là khả năng chuyển đổi một câu thành một vector số có chiều dài cố định, nắm bắt được ngữ nghĩa của câu đó. Vector này có thể được dùng cho các tác vụ khác như phân loại, tìm kiếm tương đồng, v.v.

Có hai cách phổ biến để lấy vector biểu diễn cho cả câu:

1. Lấy vector đầu ra của token [CLS] (token đặc biệt được thêm vào đầu mỗi câu).
2. Lấy trung bình cộng của các vector đầu ra của tất cả các token trong câu (Mean Pooling). Cách này thường cho kết quả tốt hơn.

Yêu cầu: Viết code để tính toán vector biểu diễn cho câu This is a sample sentence. bằng phương pháp Mean Pooling.

```
import torch
from transformers import AutoTokenizer, AutoModel

# 1. Chọn một mô hình BERT
model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

# 2. Câu đầu vào
sentences = ["This is a sample sentence."]

# 3. Tokenize câu
# padding=True: đệm câu ngắn hơn để có cùng độ dài
# truncation=True: cắt các câu dài hơn
# return_tensors='pt': trả về kết quả dưới dạng PyTorch tensors
inputs = tokenizer(sentences, padding=True, truncation=True, return_tensors='pt')

# 4. Đưa qua mô hình để lấy hidden states
# torch.no_grad() để không tính toán gradient, tiết kiệm bộ nhớ
with torch.no_grad():
    outputs = model(**inputs)

# outputs.last_hidden_state chứa vector đầu ra của tất cả các token
last_hidden_state = outputs.last_hidden_state
# shape: (batch_size, sequence_length, hidden_size)

# 5. Thực hiện Mean Pooling
# Để tính trung bình chính xác, chúng ta cần bỏ qua các token đệm (padding tokens)
attention_mask = inputs['attention_mask']
mask_expanded = attention_mask.unsqueeze(-1).expand(last_hidden_state.size()).float()
sum_embeddings = torch.sum(last_hidden_state * mask_expanded, 1)
sum_mask = torch.clamp(mask_expanded.sum(1), min=1e-9)
sentence_embedding = sum_embeddings / sum_mask

# 6. In kết quả
print("Vector biểu diễn của câu:")
print(sentence_embedding)
print("\nKích thước của vector:", sentence_embedding.shape)
```

d:\College\year6s1\nlp-dl\.venv\Lib\site-packages\huggingface_hub\file_download.py:143: UserWarning: To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. Falling back to file download.

Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to file storage.

Vector biểu diễn của câu:

```
tensor([[ -6.3875e-02, -4.2837e-01, -6.6779e-02, -3.8430e-01, -6.5785e-02,
         -2.1826e-01,  4.7636e-01,  4.8659e-01,  4.0036e-05, -7.4274e-02,
        -7.4740e-02, -4.7635e-01, -1.9773e-01,  2.4824e-01, -1.2162e-01,
         1.6679e-01,  2.1045e-01, -1.4576e-01,  1.2637e-01,  1.8635e-02,
         2.4640e-01,  5.7090e-01, -4.7014e-01,  1.3782e-01,  7.3650e-01,
        -3.3808e-01, -5.0329e-02, -1.6453e-01, -4.3517e-01, -1.2900e-01,
         1.6516e-01,  3.4004e-01, -1.4930e-01,  2.2422e-02, -1.0488e-01,
        -5.1916e-01,  3.2964e-01, -2.2162e-01, -3.4206e-01,  1.1993e-01,
        -7.0148e-01, -2.3126e-01,  1.1224e-01,  1.2550e-01, -2.5191e-01,
        -4.6375e-01, -2.7261e-02, -2.8415e-01, -9.9249e-02, -3.7019e-02,
        -8.9192e-01,  2.5005e-01,  1.5816e-01,  2.2701e-01, -2.8497e-01,
         4.5300e-01,  5.0921e-03, -7.9441e-01, -3.1008e-01, -1.7403e-01,
         4.3029e-01,  1.6816e-01,  1.0590e-01, -4.8987e-01,  3.1856e-01,
         3.2861e-01, -1.3403e-02,  1.8807e-01, -1.0905e+00,  2.1009e-01,
        -6.7579e-01, -5.7076e-01,  8.5945e-02,  1.9121e-01, -3.3818e-01,
         2.7744e-01, -4.0539e-01,  3.1305e-01, -4.1197e-01, -5.6820e-01,
        -3.9074e-01,  4.0747e-01,  9.9898e-02,  2.3719e-01,  1.0154e-01,
```

```

-2.5670e-01, -2.0583e-01, 1.1762e-01, -5.1439e-01, 4.0979e-01,
1.2149e-01, 1.9333e-02, -5.9030e-02, -2.0141e-01, 7.0860e-01,
-6.4610e-02, 2.4780e-02, -9.0588e-03, 1.9667e-02, 3.0815e-01,
-4.9832e-02, -1.0691e+00, 6.1072e-01, -4.9723e-02, -1.5156e-01,
-6.7778e-02, 4.7811e-02, 5.2103e-01, 1.6951e-01, 1.0145e-02,
5.3093e-01, -7.8189e-02, 6.5842e-02, -2.9383e-01, -4.6045e-01,
4.2071e-01, 1.1822e-01, 2.3631e-01, -4.5379e-02, -1.3740e-01,
-4.4018e-01, -6.8122e-02, 1.9934e-01, 8.7062e-01, -2.2603e-01,
3.3604e-01, 2.0236e-01, 3.7898e-01, 1.9533e-01, -3.0366e-01,
3.8633e-01, 6.1949e-01, 6.8663e-01, -1.8968e-01, -3.6815e-01,
-1.6616e-01, -7.0828e-02, -3.4610e-01, -8.5325e-01, 4.6645e-02,
2.8512e-01, 1.0890e-01, 2.5938e-01, -4.2975e-01, 4.3345e-01,
2.0637e-01, -3.8656e-01, -3.8187e-02, 3.6925e-01, 3.0130e-01,
4.0251e-01, 1.2887e-01, -3.7689e-01, -3.4447e-01, -4.2116e-01,
-1.0252e-01, -8.9736e-02, 4.7384e-01, 8.1716e-02, 1.5885e-01,
7.6674e-01, 3.4493e-01, 9.8530e-04, 4.8932e-02, 2.6132e-01,
3.8329e-02, -2.0035e-01, 2.6654e-01, 9.3774e-02, -4.6780e-02,
-4.0519e-01, -4.4310e-01, 6.1269e-01, -1.8950e-01, -3.8333e-01,
2.0583e-01, 1.5379e-01, -1.4664e-01, 5.3847e-01, -3.9618e-01,
-2.0599e+00, 6.7053e-01, 2.1112e-01, -4.7306e-01, 3.4865e-01,
-2.9919e-01, 5.4614e-01, -5.3925e-01, -2.4877e-01, -2.9069e-02,
-2.0319e-01, -7.3275e-02, -3.8147e-01, -5.4455e-01, 3.5050e-01,
-1.1249e-01, -2.1472e-01, -3.8439e-01, -1.0760e-01, -8.8820e-02,
2.5263e-01, 2.1448e-01, 5.5798e-02, -6.5411e-02, 9.9838e-02,
3.3435e-01, 2.4018e-01, 2.9876e-02, -1.1191e-01, 5.4330e-01,
-5.5214e-01, 1.1125e+00, 5.4141e-01, -7.4160e-02, 3.5337e-01,
1.2313e-01, 3.4856e-02, -2.8568e-01, -1.2517e-01, -4.4332e-02,
1.3323e-01, -2.4996e-01, -4.9834e-01, 4.1959e-01, -3.1580e-01,
6.1942e-01, 3.1113e-01, 4.8846e-01, 6.1518e-01, -3.6327e-02,
2.1295e-02, -3.5715e-01, 5.9126e-01, 1.5102e-01, -2.9641e-01,
2.9441e-01, -1.4139e-01, 1.1662e-01, -3.6223e-01, -1.4621e-01,
6.5255e-02, 3.9270e-01, 3.8543e-01, -2.3996e-01, -3.1482e-01,
-4.6860e-01, -1.1920e-01, 8.6234e-02, -3.4597e-02, -3.6275e-01,
-3.9838e-01, -3.6006e-01, -1.9672e-01, -2.7738e-01, -4.1097e-01,
3.6456e-01, -2.6012e-01, 1.2587e-01, 1.2752e-01, 5.4261e-01,

```

Câu hỏi:

1. Kích thước (chiều) của vector biểu diễn là bao nhiêu? Con số này tương ứng với tham số nào của mô hình BERT?
 - Kích thước của vector biểu diễn là [1,768]
 - Con số này tương ứng với tham số Hidden embedding size chính là số chiều vector mà bert sử dụng để biểu diễn embedding khi token đi qua các encoder layer.
2. Tại sao chúng ta cần sử dụng attention_mask khi thực hiện Mean Pooling?
 - Mean-Pooling là lấy mean embedding của các token
 - Các token là các câu có thể có độ dài ngắn khác nhau => Cần có padding
 - attention-masks là phương pháp bỏ qua các padding khi áp dụng mean pooling
 - Vậy ta cần sử dụng attention-mask khi thực hiện mean pooling vì các padding với các giá trị 0 có thể làm lệch giá trị mean tính được.

